Ecole Supérieure Privée
d'Ingénierie et de Technologies

Private Higher School of Engineering and Technologies (ESPRIT)

Integration Project Report

Project Title:

Intelligent platform for managing training programs and professional certifications

SkillHub Pro

Presented by:

- Khalil Dridi
- Manef Akrimi
- Jasser Noomani
- Skander Ferjeni
- Seif Ayed
- Ayoub Bel Gacem

Academic Year 2025/2026

Contents

1 Project Framework Presentation **2**

1.1 General framework of the work 2

1.1.1 Problem Statement 2

1.1.2 Description of the Work to Be Done 3

1.2 Study of Existing Solutions 3

1.2.1 Description of the Existing Situation 3

1.2.2 Critique of the Existing Situation 4

1.2.3 Proposed Solution 5

1.3 Methodological Choices and Adopted Formalism 6

1.3.1 Comparative Study between Classical and Agile Methods 6

1.3.2 Adopted Formalism 7

2 Planning and System Architecture **9**

2.1 Stakeholders and Actors Identification 9

2.2 Requirements Analysis 10

2.2.1 Functional Requirements 10

2.2.2 Non-Functional Requirements 10

2.3 System Modeling (UML) 11

2.3.1 Use Case Diagram 11

2.3.2 Class Diagram 13

2.4 System Architecture Design 14

2.4.1 Global Architecture Overview 14

2.4.2 Microservices Architecture 14

2.4.3 Communication Architecture 15

2.4.4 Architecture Patterns 15

2.5 Machine Learning Architecture 15

2.5.1 Recommendation System Design 15
2.5.2 Data Flow 15
2.5.3 ML Integration in Microservices 15
2.6 DevOps Architecture 16
2.6.1 DevOps Strategy 16
2.6.2 CI/CD Pipeline Design 16
2.6.3 Containerization (Docker) 16
2.6.4 Orchestration (Kubernetes) 16
2.6.5 Deployment Architecture 16
2.7 Project Planning (Agile Scrum) 17
2.7.1 Product Backlog 17
2.7.2 Sprint Planning 17
2.7.3 Roadmap 17
2.8 Development Environment 17
2.8.1 Hardware Environment 17
2.8.2 Software Environment 17
2.8.3 Technology Stack (example) 18

List of Figures

1.1 Comparison between classical and agile methods 6

1.2 Scrum framework: roles, artifacts and ceremonies 7

2.1 Global Use Case Diagram of SkillHub Pro 12

2.2 UML Class Diagram – Analysis Model of the Training Management Platform . 13

List of Tables

General Introduction

In the digital era, continuous learning and skill development have become essential to meet the demands of a constantly evolving labor market. Both organizations and individuals face the need to acquire new skills to adapt to technological, economic, and social transformations.

In this context, training platforms play a central role in providing access to knowledge and enhancing professional competencies. However, traditional training management systems often have limitations in terms of personalization, efficiency, and intelligent data utilization. Learners sometimes struggle to identify the most suitable courses for their needs, while organizations face challenges in managing and monitoring training programs.

To address these issues, this integration project aims to design and develop an intelligent platform for managing professional training and certifications. This platform will centralize the management of courses, users, enrollments, and assessments while integrating an intelligent recommendation module based on Machine Learning technques. The goal is to provide a more efficient, personalized, and scalable digital environment that meets the expectations of both learners and organizations.

This report presents the various stages of project design and planning, focusing on needs analysis, system modeling, adopted architecture, and development planning. It thus provides a solid foundation for the technical implementation of the platform.

Chapter 1

Project Framework Presentation

Introduction

This chapter aims to present the general framework of the integration project. It provides the context in which the project is carried out, outlines the problem it addresses, and describes the work to be completed. A review of existing solutions is also presented to identify their limitations and justify the need for the proposed solution. Finally, this chapter introduces the chosen methodology and the modeling formalism used for system design. This chapter also prepares the reader for the following chapters, where the technical architecture, microservices design, and DevOps pipeline will be presented in detail.

1.1 General framework of the work

The digitalization of training and certification processes has become essential to meet the growing demand for professional skills. Digital platforms now make it easier to manage training programs and improve learner support.

In this context, an intelligent platform for managing training and certifications is proposed to optimize the organization of training pathways and provide recommendations tailored to users' profiles. The platform will also include a recommendation module based on Machine Learning to suggest the most relevant courses. It aims to offer a scalable and efficient solution that meets the needs of both learners and organizations.

1.1.1 Problem Statement

Learners often struggle to find training programs that match their skills, while organizations face challenges in effectively monitoring training pathways and certifications. Therefore, it is necessary to provide a solution capable of managing training programs while automatically recommending the most relevant courses. This recommendation will be personalized based on each learner's profile and skills.

1.1.2 Description of the Work to Be Done

The project involves designing an intelligent platform to manage training programs, users, enrollments, and certifications. The system will also include a recommendation module to suggest the most suitable courses for each learner. The minimum viable product (MVP) will cover user management, course management, enrollments, and basic reporting, with the recommendation module as a core feature.

1.2 Study of Existing Solutions

1.2.1 Description of the Existing Situation

Currently, several platforms are used for managing training programs and professional certifications. The most well-known are Moodle, Coursera, and Udemy, each with its own characteristics.

Moodle: a widely used open-source platform in educational institutions and companies. It allows the creation of online courses, user management, assessments, and discussion forums.

Coursera: a commercial platform offering online courses and certificates issued by partner universities or institutions. It provides an intuitive interface and a large course catalog.

Udemy: a commercial platform aimed at the general public, with a diverse catalog and features for tracking progress and issuing certificates.

Main Features

- User and enrollment management
- Course publication and access
- Assessment tracking and certificate issuance
- Access to online learning resources

Technologies Used

- **Moodle**: PHP, MySQL, JavaScript
- **Coursera and Udemy**: primarily modern web architectures (Java, Python, web frameworks), relational databases, and basic recommendation systems for some content

This review highlights the functionalities offered by current platforms while setting the stage for discussing their limitations and the need for a more intelligent and adaptable solution.

1.2.2 Critique of the Existing Situation

Although existing platforms such as Moodle, Coursera and Udemy provide useful features for managing training programs and certifications, they present several limitations that reduce their effectiveness and adaptability to the current needs of learners and organizations.

Technical Limitations

- **Limited personalization**: these platforms do not consistently provide intelligent recommendations tailored to each user's profile, skills and objectives.
- **Sometimes complex interfaces**: some platforms, particularly Moodle, may appear unintuitive for new users, which slows adoption and daily use.
- **Restricted tracking features**: analytics tools for monitoring learning paths are often basic and do not enable detailed tracking of progress and performance.

Pedagogical and Organizational Limitations

- **Insufficient pedagogical tracking**: instructors and training managers have limited tools to adapt learning paths to individual needs.
- **Lack of advanced reporting**: the statistics provided by current platforms are limited, making it difficult to support data-driven decisions to improve course quality.
- **Rigid content management**: configuring and updating courses often requires significant technical skills, which can be an obstacle for smaller organizations.

Justification of the Project

These limitations highlight the need for a more intelligent and adaptable solution capable of:

- offering personalized recommendations for each learner,
- simplifying the administration and monitoring of training programs,
- providing advanced reporting and analytics to improve pedagogical quality.

By addressing these needs, the proposed project aims to fill gaps in existing solutions and deliver clear added value to users and institutions.

1.2.3 Proposed Solution

To address the identified shortcomings, the proposed solution is to develop an intelligent platform **SkillHub Pro** for managing training and professional certifications that is adapted to the needs of learners and organizations.

Key Components of the Solution

- **Intelligent course recommendation**: a module based on Machine Learning techniques will automatically suggest the most relevant courses according to each user's profile, skills and goals.
- **Personalized progress tracking**: training managers will be able to monitor learner progress in near real time and adapt learning paths to specific needs.
- **Intuitive and ergonomic interface**: the platform will be designed for ease of use by learners, instructors and administrators alike.
- **Advanced reporting tools**: dashboards and detailed reports will help analyze performance, identify weaknesses and improve course quality.
- **Flexibility and scalability**: the platform will support the integration of new features and adapt to various organizational contexts and types of training.

Added Value Compared to Existing Platforms

- **Personalized learning paths**: unlike current solutions, each learner will receive recommendations tailored to their needs and competencies.
- **Improved pedagogical monitoring**: instructors and managers will be able to steer training more effectively using detailed analytics.
- **Better user experience**: a clear and intuitive interface will increase learner engagement and success.
- **Data-driven decisions**: managers will have reliable data to continuously refine content and training management.

In summary, the proposed solution offers an intelligent, personalized and user-centered approach that addresses the shortcomings of existing platforms and brings significant added value to professional training.

1.3 Methodological Choices and Adopted Formalism

1.3.1 Comparative Study between Classical and Agile Methods

Classical methods, such as the Waterfall model, follow a linear and sequential process where each phase must be completed before moving to the next one. This lack of flexibility makes them less suitable for projects in which requirements may evolve over time

In contrast, Agile approaches rely on iterative and incremental cycles that promote continuous feedback, close collaboration with stakeholders, and frequent delivery of functional product increments.

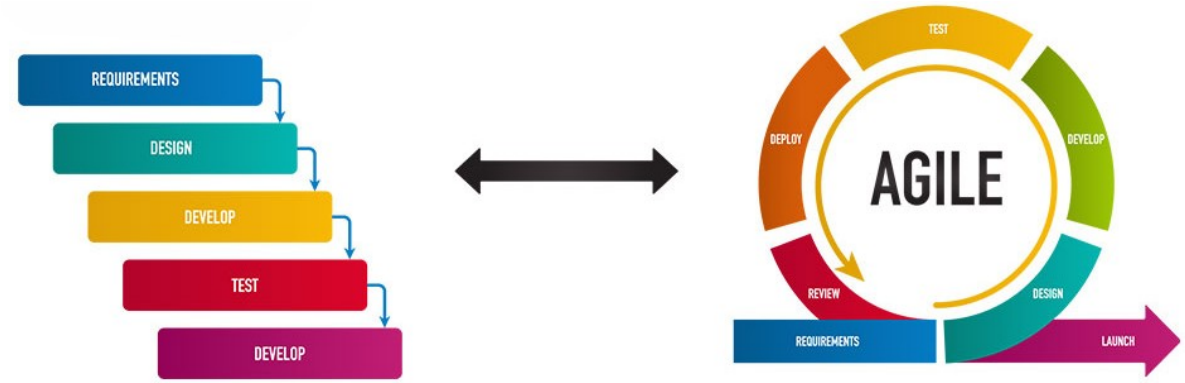

Figure 1.1: Comparison between classical and agile methods

1.3.1.1 Why Scrum?

Scrum was chosen for its ability to manage complex and evolving projects through short development cycles (sprints) and regular reviews with stakeholders. This approach improves alignment with expectations, facilitates early feedback, and reduces the risk associated with changing requirements.

1.3.1.2 Scrum Framework

The Scrum framework organizes work into fixed-length iterations called *sprints* (we propose 2-week sprints). Each sprint delivers a potentially shippable increment. Key Scrum artifacts include the Product Backlog, the Sprint Backlog and the Definition of Done. Ceremonies comprise Sprint Planning, Daily Scrum, Sprint Review and Sprint Retrospective. These practices promote transparency, inspection and adaptation throughout the development process.

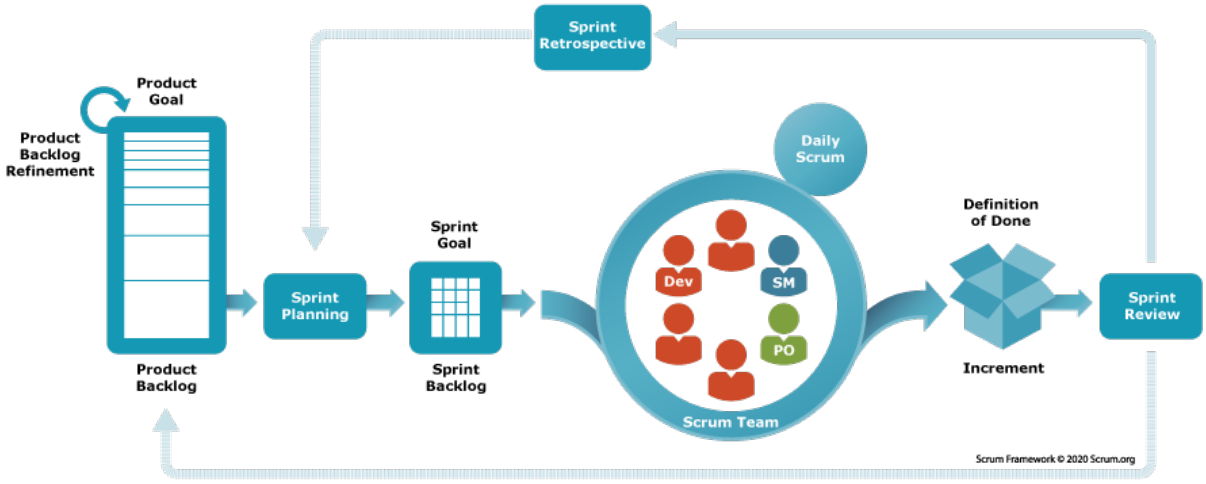

Figure 1.2: Scrum framework: roles, artifacts and ceremonies

1.3.1.3 The Scrum Team

- **Product Owner**: responsible for defining priorities and maintaining the product backlog.
- **Development Team**: implements features and ensures the quality of deliverables.
- **Scrum Master**: facilitates the Scrum process, removes impediments and protects the team from external disruptions.

For this project, roles will be assigned among the six team members. A suggested distribution is: 1 Product Owner, 1 Scrum Master (combined with DevOps responsibilities), 3 Developers (backend/frontend), and 1 Data Scientist / QA engineer. The final assignment should be confirmed by the team.

1.3.2 Adopted Formalism

To model the platform, we will use UML, which allows clear representation of requirements and design through multiple diagram types. The report will include:

- Use case diagrams,
- Class diagrams,
- Sequence diagrams,
- Component diagrams,
- Deployment diagram.

Conclusion

In summary, Scrum and UML have been selected to support iterative delivery and clear system modeling. Chapter 2 will present the detailed architecture, microservices design and the project plan, including sprint schedules and the CI/CD pipeline.

Chapter 2

Planning and System Architecture

Introduction

This chapter details the planning and the system architecture of the project. It describes the stakeholders and requirements, presents UML models used for specification, and defines the technical architecture at both high and component level. We also present the design of the Machine Learning recommendation module and the DevOps strategy (CI/CD, containerization, orchestration). Finally, a sprint plan and the development environment are proposed to support implementation.

2.1 Stakeholders and Actors Identification

The main stakeholders and actors involved in the SkillHub Pro platform are:

- **Visitor**: a non-authenticated user who can consult the course catalog and register on the platform.
- **Learner**: an authenticated user who can enroll in courses, manage their profile, follow learning progress, take assessments, download certificates, receive notifications, and obtain personalized course recommendations.
- **Instructor**: responsible for creating and managing course content, managing assessments, and monitoring learners' progress through analytics dashboards.
- **Administrator**: responsible for managing users (learners and instructors), roles and permissions, and accessing global reports for platform supervision.
- **Recommendation Engine**: a system component that generates personalized course recommendations based on learners' profiles, skills, and learning history.

2.2 Requirements Analysis

2.2.1 Functional Requirements

Key functional requirements include:

- User management: registration, authentication, profile & roles.
- Course management: create/edit courses, modules, materials (videos, PDFs, quizzes).
- Enrollment management: enrollments, progress tracking, completion.
- Assessment and certification: quizzes/exams, grading, certificate issuance.
- Recommendation engine: personalized course suggestions.
- Reporting & dashboards: analytics for instructors and administrators.
- Notifications: email/push notifications for events.
- API access: REST endpoints for frontend and integrations.

2.2.2 Non-Functional Requirements

Non-functional constraints and expectations:

- **Scalability**: horizontally scalable services.
- **Availability**: high availability for critical services.
- **Security & Privacy**: secure authentication (JWT/OAuth2), password hashing, data encryption and GDPR-aware handling.
- **Performance**: acceptable response times for core APIs.
- **Maintainability**: modular microservices, automated tests.
- **Observability**: centralized logs, metrics and tracing.

2.3 System Modeling (UML)

2.3.1 Use Case Diagram

A high-level use case diagram will show interactions between actors and main use cases .

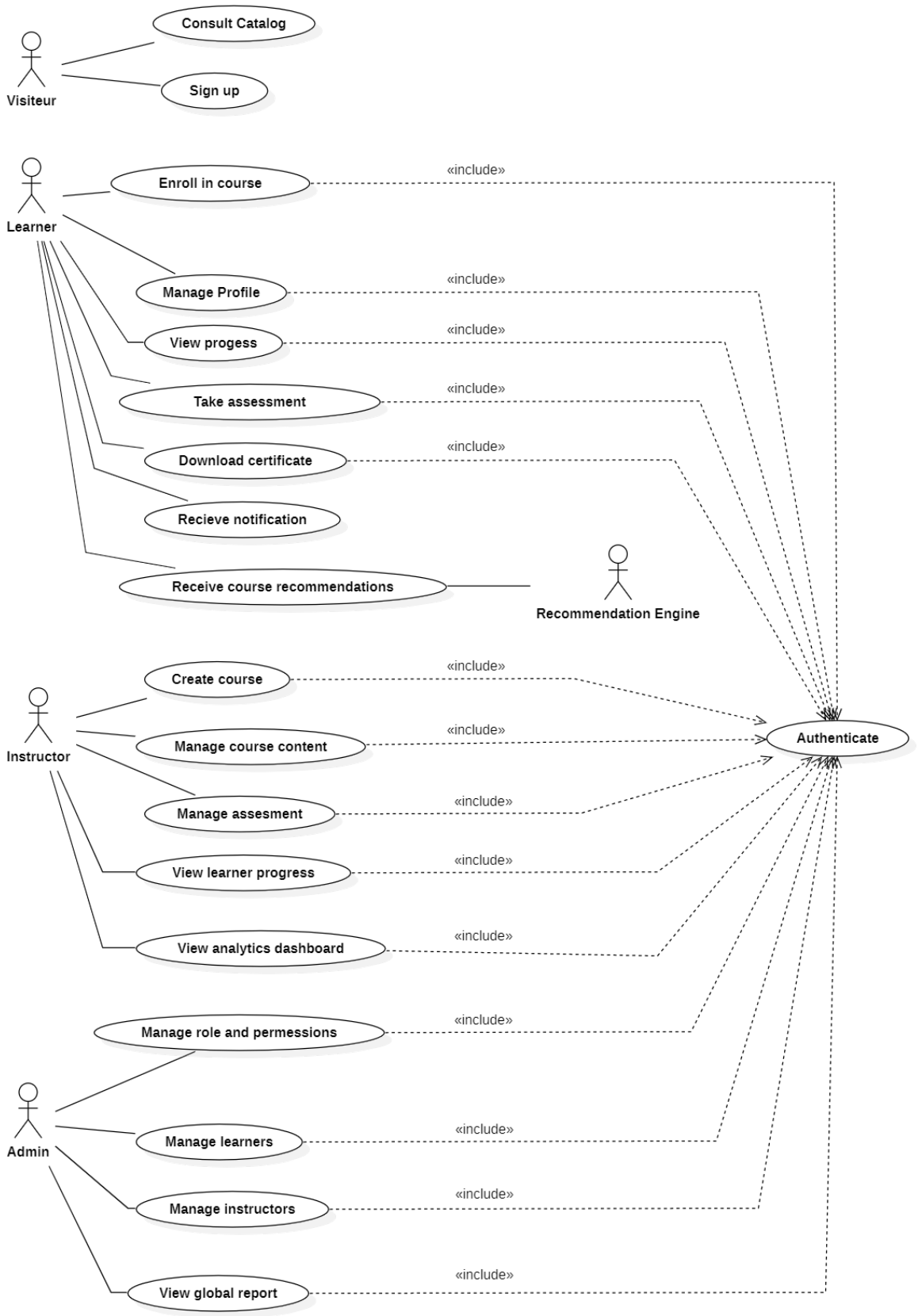

Figure 2.1: Global Use Case Diagram of SkillHub Pro

2.3.2 Class Diagram

Define domain classes: `User`, `Profile`, `Course`, `Module`, `Enrollment`, `Assessment`, `Certificate`, `Recommendation`, `Skill`, with relationships and key attributes.

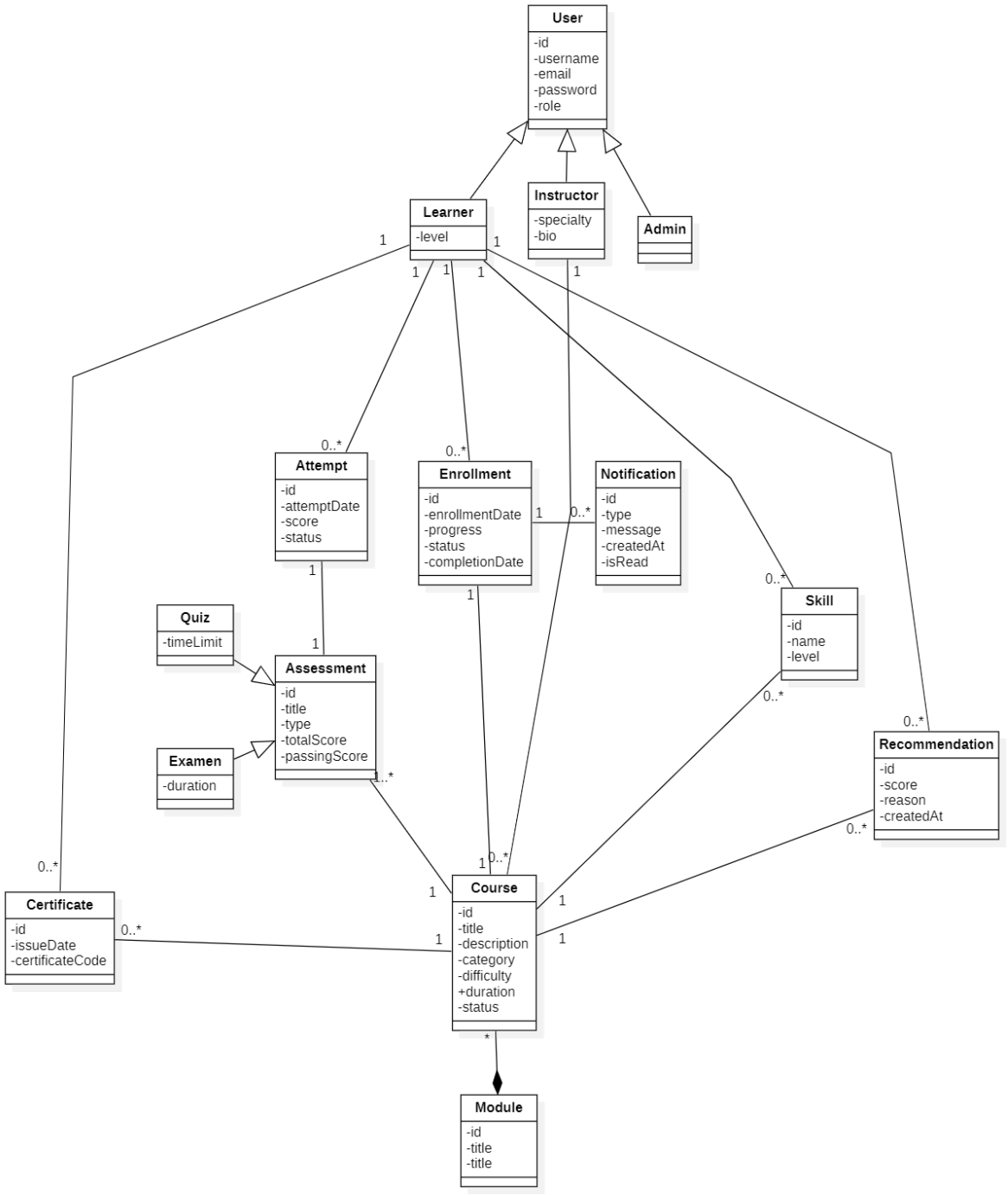

Figure 2.2: UML Class Diagram – Analysis Model of the Training Management Platform

2.4 System Architecture Design

2.4.1 Global Architecture Overview

The system adopts a microservices architecture. Services communicate via REST for synchronous operations and via a message broker (RabbitMQ or Kafka) for asynchronous events. Cross-cutting layers include an API Gateway, service discovery, data storage per service, and centralized observability.

2.4.2 Microservices Architecture

Microservices Identification

Suggested services:

- Auth Service
- User Service
- Course Service
- Enrollment Service
- Recommendation Service
- Assessment Service
- Certification Service
- Notification Service
- Analytics Service
- API Gateway

Responsibilities of Each Service

Each service encapsulates domain logic, data access and API. Services expose REST endpoints (OpenAPI) and publish domain events when needed.

Database per Service

Each service owns its datastore to avoid coupling. Example: PostgreSQL for User and Course services, Redis for caching / feature store, object storage (S3 / MinIO) for large artifacts.

2.4.3 Communication Architecture

Synchronous interactions use REST APIs. Asynchronous events are handled via a message broker (RabbitMQ/Kafka) for notifications, analytics and background processing.

2.4.4 Architecture Patterns

- **API Gateway**: routing, TLS termination, authentication.
- **Service Discovery**: dynamic resolution of service endpoints.
- **Load Balancer**: distribute incoming traffic.
- **Resilience patterns**: circuit breakers, retries and bulkheads.

2.5 Machine Learning Architecture

2.5.1 Recommendation System Design

We propose a hybrid recommender combining content-based and collaborative approaches. Optionally, embedding-based similarity can be used for richer semantic recommendations.

2.5.2 Data Flow

- Event ingestion: publish user events to the broker.
- ETL: consume events to build training datasets.
- Feature store: store online features in Redis and offline features in object storage.
- Model training: offline training and evaluation (Precision@K, NDCG).
- Model serving: expose inference endpoints through the Recommendation Service.
- Retraining: scheduled or triggered retraining strategies.

2.5.3 ML Integration in Microservices

The Recommendation Service exposes endpoints such as `/recommendations?userId=`. Services call this API and cache results to reduce latency.

2.6 DevOps Architecture

2.6.1 DevOps Strategy

Adopt Infrastructure as Code (Terraform), environment separation (dev / staging / prod) and automated pipeline for builds and deployments.

2.6.2 CI/CD Pipeline Design

Pipeline per service:

1. Build and unit tests.
2. Containerization (Docker).
3. Integration tests in ephemeral environments.
4. Security and dependency scans.
5. Publish image to registry.
6. Deploy to staging and production (Helm/Kubernetes).

2.6.3 Containerization (Docker)

Use multi-stage Docker builds and tag images with commit SHAs for traceability.

2.6.4 Orchestration (Kubernetes)

Use Kubernetes resources (Deployments, Services, ConfigMaps, Secrets) and Helm charts for release management. Implement autoscaling and rolling updates.

2.6.5 Deployment Architecture

Deploy to cloud-managed Kubernetes or on-prem cluster. Use managed databases and object storage for reliability. Adopt blue/green or canary deployment strategies for safe releases.

2.7 Project Planning (Agile Scrum)

2.7.1 Product Backlog

Maintain a prioritized backlog with epics and user stories (user management, course CRUD, enrollment, recommendation, analytics, etc.).

2.7.2 Sprint Planning

Recommend 2-week sprints. Example sprint breakdown:

- Sprint 1: project setup, Auth and User service, frontend skeleton.
- Sprint 2: Course Service and course catalog UI.
- Sprint 3: Enrollment Service, progress tracking and notifications.
- Sprint 4: Recommendation Service (POC) and offline model evaluation.
- Sprint 5: Assessment and Certification Services, reporting.
- Sprint 6: DevOps stabilization, CI/CD and monitoring.

2.7.3 Roadmap

Map sprints to milestones: MVP, Beta, Release. Reserve time for integration and testing.

2.8 Development Environment

2.8.1 Hardware Environment

Developers will use modern developer machines; CI uses cloud runners; staging/prod uses cloud-managed infrastructure.

2.8.2 Software Environment

- Backend: Spring Boot (Java) s.
- Recommendation: .

- Databases: PostgreSQL, H2, mysql.
- Messaging: Kafka.
- Containerization: Docker; Orchestration: Kubernetes.
- CI/CD: GitHub Actions ; IaC: Terraform; Monitoring: Prometheus & Grafana.

2.8.3 Technology Stack (example)

Languages: Java + Python; Frontend: React; Datastores: PostgreSQL, Redis; Messaging: RabbitMQ/Kafka; Container: Docker; Orchestration: Kubernetes; CI/CD: GitHub Actions.

Conclusion

This chapter defined a modular microservices architecture, a practical ML integration design and a DevOps strategy to enable continuous delivery. The proposed planning and technical choices balance short-term delivery (MVP) with long-term maintainability and scalability. The next chapter will detail the API contracts, data models and implementation specifics for each microservice.